

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
СКОЛЬКО РАЗ ТЫ СКАЗАЛ СЛОВО “ПОТОМ”.»
Вариант 22

Студент:

И.А. Михалёнок

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – освоить практические приёмы работы в интегрированной среде разработки IntelliJ IDEA при создании объектно-ориентированного Java-приложения с графическим интерфейсом, изучить процессы сборки, запуска и отладки программ, а также освоить проектирование приложений на основе архитектурного паттерна MVC с активной моделью.

В соответствии с вариантом 22 требуется разработать утилиту «Сколько раз ты сказал "потом"», которая по введённому пользователем возрасту оценивает примерное количество случаев, когда человек откладывал дела на потом, используя усреднённые статистические данные. Для выполнения работы поставлены следующие задачи:

1. Создать проект Java в IntelliJ IDEA, настроить, изучить основные средства редактирования кода, навигации, сборки, запуска и отладки приложения.
2. Установить и настроить плагины IntelliJ IDEA для работы с UML-диаграммами и средствами описания архитектуры в нотации SysML v2, отразить процесс настройки в отчёте.
3. Создать главную форму приложения с кнопкой «Ввести данные» и отдельное модальное окно для ввода возраста пользователя.
4. Обеспечить запоминание последнего корректно введённого значения возраста и его автоматическое восстановление при повторном открытии окна ввода.
5. Реализовать проверку вводимых значений и вывод понятного сообщения об ошибке при некорректном вводе.
6. Разделить приложение на три компонента: модель, представление и контроллер. Реализовать активную модель через паттерн «Наблюдатель», при котором модель самостоятельно уведомляет представление об изменении состояния.
7. Проверить основные сценарии работы программы (корректный ввод, восстановление данных, обработка ошибок), подготовить ответы на контрольные вопросы и оформить отчёт.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Современная интегрированная среда разработки представляет собой комплекс программных средств, объединённых в едином интерфейсе для повышения производительности труда программиста. В типичный состав IDE входят: текстовый редактор с подсветкой синтаксиса, компилятор или интерпретатор, система автоматической сборки проекта, отладчик с точками останова, встроенный терминал, модуль интеграции с системами контроля версий, а также средства навигации по коду и автоматического рефакторинга.

2.2 Редактирование и навигация в IntelliJ IDEA

Среда предоставляет широкий набор инструментов для ускоренной работы с исходным кодом. К основным возможностям редактора относятся: интеллектуальное автодополнение (Ctrl+Space), автоматическая вставка импортов, быстрое исправление ошибок через Alt+Enter, форматирование кода по стандарту (Ctrl+Alt+L), а также безопасное переименование идентификаторов (Shift+F6), при котором изменения автоматически применяются ко всем связанным файлам.

Для перемещения по проекту используются: переход к объявлению символа (Ctrl+B), поиск всех использований (Alt+F7), просмотр иерархии классов (Ctrl+N) и структурная панель проекта (Project). Результаты компиляции и работы программы отображаются в окнах Build и Run, а предупреждения анализатора – в окне Problems.

2.3 Сборка, запуск и отладка Java-приложения

Исходные тексты программы на языке Java преобразуются компилятором javac в байт-код, исполняемый виртуальной машиной JVM. В IntelliJ IDEA сборка проекта выполняется командой Build Project, а для запуска приложения создаётся конфигурация запуска, указывающая главный класс с методом main.

Отладка программы производится в специальном режиме Debug. Точка останова устанавливается щелчком мыши в левом поле редактора или сочетанием клавиш Ctrl+F8. При достижении точки останова выполнение приостанавливается, и в окне Debug отображаются стек вызовов, значения локальных переменных и полей объектов. Пошаговое выполнение осуществляется командами Step Over (перешагнуть), Step Into (войти в метод) и Step Out (выйти из метода). Для вычисления произвольных выражений в контексте текущего кадра используется Evaluate Expression.

2.4 Паттерн MVC

MVC (Model-View-Controller) – это архитектурный шаблон проектирования, согласно которому приложение разделяется на три слабосвязанных компонента. Модель (Model) инкапсулирует данные предметной области и бизнес-логику, не завися от способа отображения. Представление (View) отвечает за визуальное представление данных и взаимодействие с пользователем. Контроллер (Controller) принимает пользовательские команды, интерпретирует их и передаёт управление модели или представлению.

Такое разделение ответственности снижает связанность классов, упрощает тестирование бизнес-логики независимо от графического интерфейса и облегчает дальнейшее сопровождение и модификацию приложения.

2.5 Активная и пассивная модель

Различают два подхода к организации взаимодействия модели и представления.

При пассивной модели модель является простым хранилищем данных и не предпринимает никаких действий при их изменении. Обязанность по обновлению интерфейса ложится на контроллер, который после каждой операции модификации данных вручную вызывает методы обновления представления.

При активной модели модель реализует механизм оповещения подписчиков. Представление регистрируется в модели в качестве слушателя и автоматически получает уведомление при каждом изменении состояния.

2.6 Проверка корректности ввода данных

Для предотвращения сбоев при работе с пользовательским вводом применяется многоуровневая проверка. На первом этапе выполняется попытка преобразования строки в целое число с перехватом исключения `NumberFormatException`. На втором этапе проверяется соответствие полученного значения допустимому диапазону. При обнаружении ошибки формируется текстовое сообщение, которое передаётся в представление и отображается пользователю через диалоговое окно.

2.7 Сохранение и восстановление последних данных

Механизм запоминания последнего корректного ввода реализуется путём хранения соответствующего значения в контроллере. Перед открытием диалогового окна ввода это значение подставляется в текстовое поле, что избавляет пользователя от повторного набора и снижает вероятность ошибки. При успешной обработке новых данных сохранённое значение обновляется.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработана утилита «Сколько раз ты сказал “потом”». Пользователь вводит возраст, после чего программа оценивает примерное количество откладываний дел за жизнь.

Расчёт выполняется по формуле:

$$N = \text{возраст} \times 365,25 \times 3,$$

где N – оценочное количество раз, когда человек мог сказать «потом». Значение 365,25 учитывает високосные годы.

Возраст принимается в диапазоне от 1 до 120 лет. При корректном вводе результат сохраняется в модели. При повторном открытии окна ввода последний корректный возраст автоматически подставляется.

3.2 Настройка среды и расширений для UML и SysMLv2

Проект разработан на языке Java в IntelliJ IDEA. В качестве Project SDK рекомендуется использовать JDK 21.

Для UML может использоваться плагин PlantUML Integration. Диаграмма MVC подготовлена в файле src/mvc.puml. Для SysMLv2 в отчёте представлено текстовое структурное описание компонентов проекта и использовано совместимое средство моделирования, доступное в Settings – Plugins.

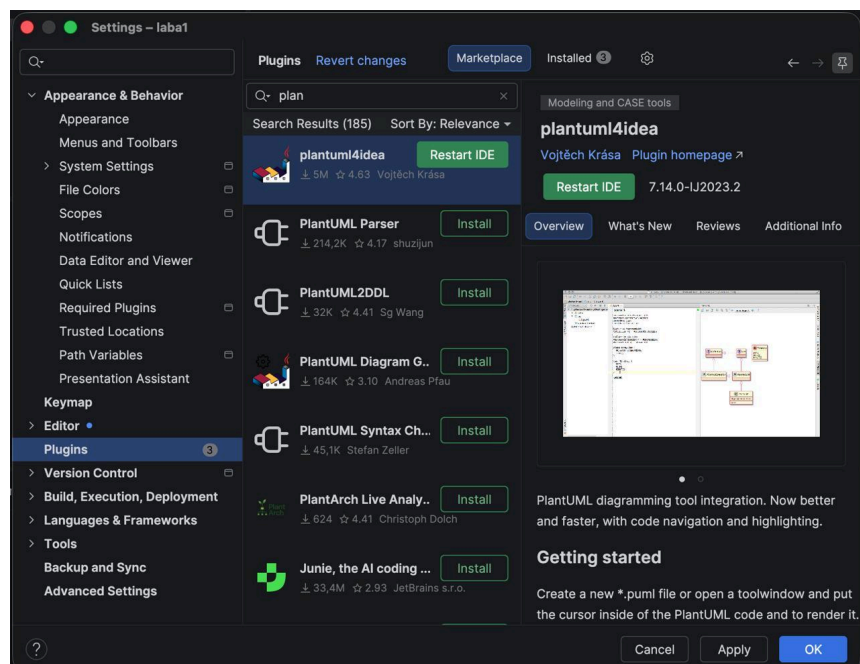


Рисунок 3.1 – Настройка проекта и средства моделирования IntelliJ IDEA

Структурное представление системы в нотации:

```
package ProcrastinationApp {
    part def PostponementModel;
    part def MainView;
    part def InputDialog;
    part def PostponementController;

    part application {
        part model : PostponementModel;
        part view : MainView;
        part dialog : InputDialog;
        part controller : PostponementController;

        connect view to controller;
        connect controller to model;
        connect model to view;
    }
}
```

3.3 Демонстрация отладчика

Работа отладчика демонстрируется в методе `setAge()` класса `PostponementModel`. Точка останова устанавливается на строке вычисления `estimatedPostponements`:

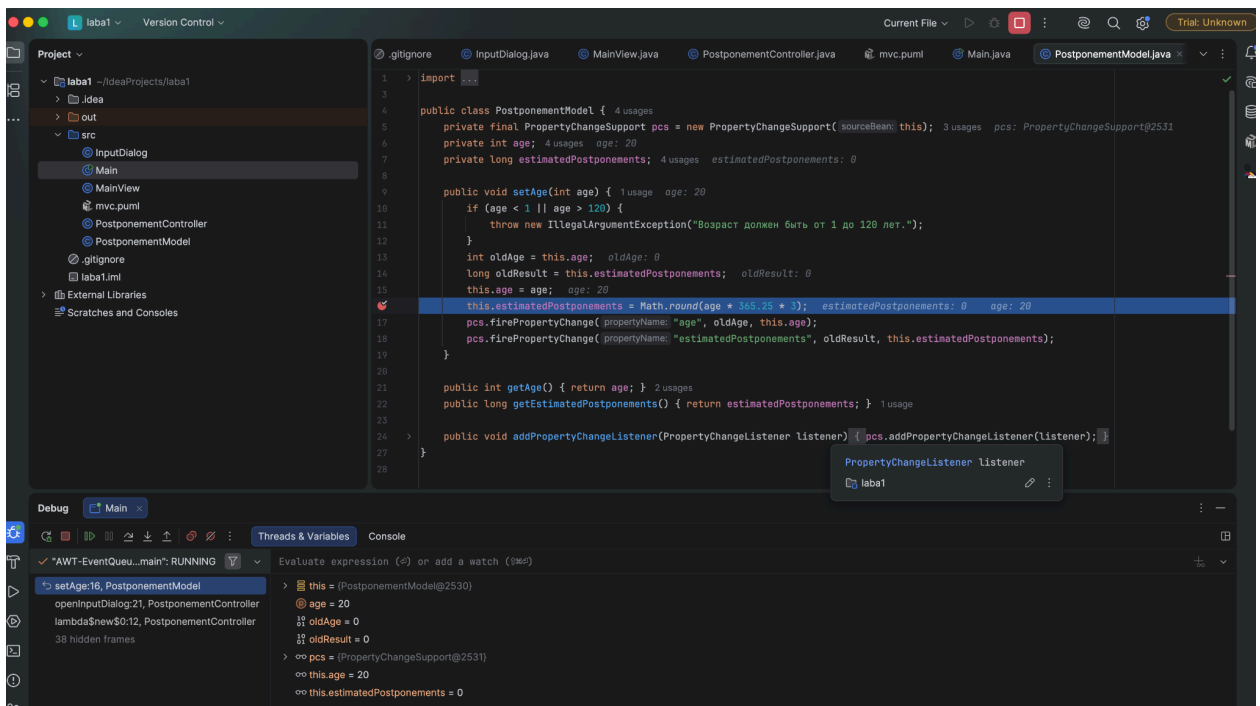


Рисунок 3.2 – Остановка программы в отладчике IntelliJ IDEA

После запуска программы в режиме Debug (Shift+F9) и ввода возраста в диалоговом окне выполнение приостанавливается на указанной строке. В

окне Debug отображается стек вызовов, значения переменных `age`, `oldAge`, `oldResult`, а также значение константы `POSTPONEMENTS_PER_DAY`. С помощью команды `Step Over` можно проследить, как вычисляется итоговое значение `estimatedPostponements`. Функция `Evaluate Expression` позволяет вычислить произвольное выражение, например $age \times 365.25 \times 3$, и сравнить его с результатом работы программы.

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- `PostponementModel` – модель, содержащая возраст, результат расчёта и механизм уведомления слушателей;
- `MainView` – главное окно, отображающее возраст и результат; реализует `PropertyChangeListener`;
- `InputDialog` – модальное окно для ввода возраста;
- `PostponementController` – контроллер, открывающий окно ввода, преобразующий строку в число и передающий значение модели;
- `Main` – точка входа, создающая компоненты MVC и запускающая интерфейс.

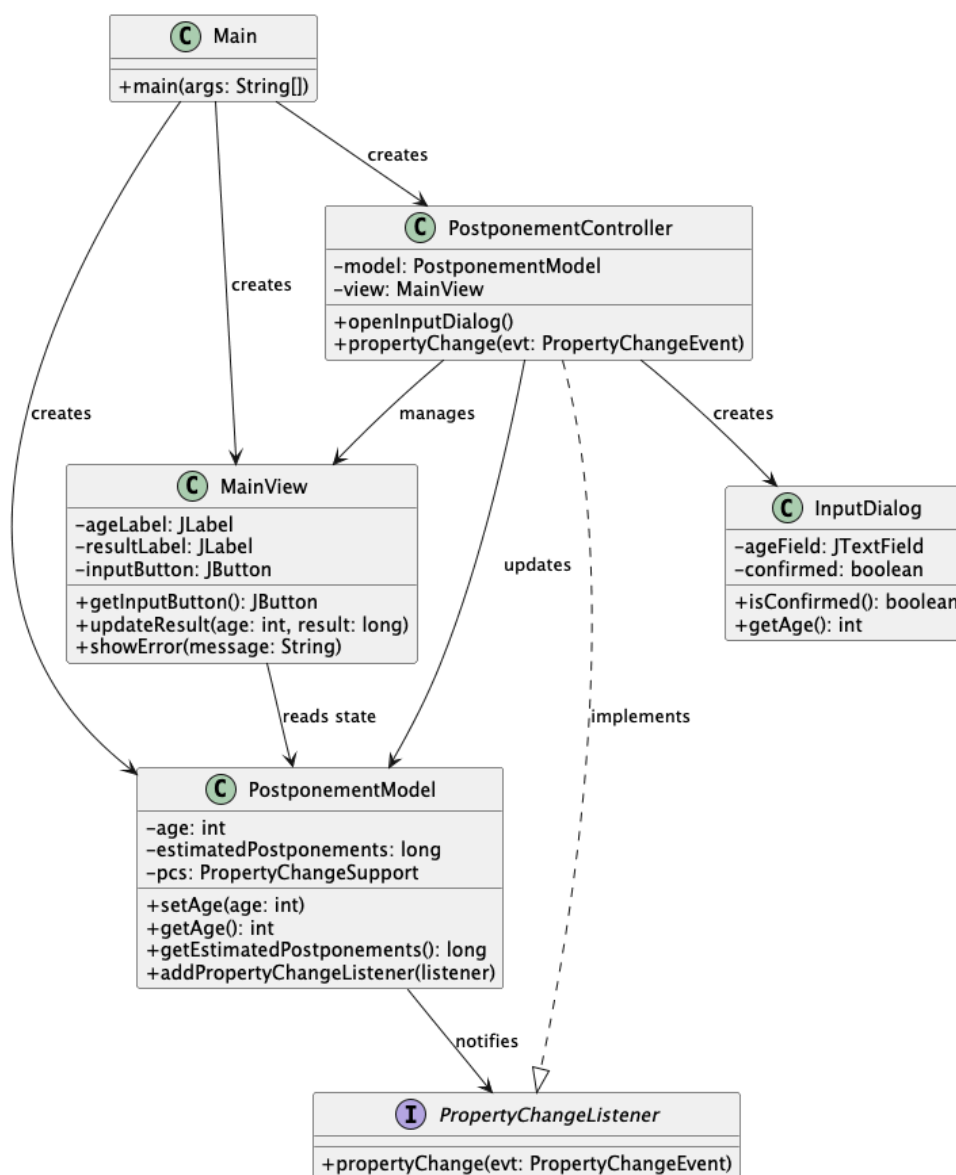


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

Класс `PostponementModel` реализует активную модель. Метод `setAge()` проверяет диапазон возраста, выполняет расчёт и генерирует `PropertyChangeEvent`. Для рассылки событий используется `PropertyChangeSupport`.

После успешной проверки значение возраста записывается в поле `age`, а результат вычисляется по формуле $age \times 365,25 \times 3$. Затем модель уведомляет об изменении состояния.

3.6 Представление

`MainView` содержит заголовок, информацию о возрасте, результат расчёта и кнопку «Ввести данные». Представление подписывается на модель

в конструкторе. Метод `propertyChange()` вызывается автоматически после изменения модели и обновляет текстовые метки.

`InputDialog` представляет отдельное модальное окно с текстовым полем возраста и кнопкой «Рассчитать». Для удобства нажатие `Enter` в поле также подтверждает ввод.

3.7 Контроллер

`PostponementController` управляет открытием `InputDialog` и обработкой пользовательского ввода. Строковое значение преобразуется в `int`. `NumberFormatException` обрабатывается отдельным сообщением, а `IllegalArgumentException` используется для ошибок диапазона возраста.

3.8 Запоминание данных и обработка ошибок

Последнее корректное значение хранится в `PostponementModel`. Перед открытием окна контроллер вызывает `setAge()` у `InputDialog`, поэтому ранее введённый возраст восстанавливается. Некорректные значения не записываются в модель и не изменяют результат на главной форме.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В IntelliJ IDEA создано Java-приложение с графическим интерфейсом. Программа принимает возраст пользователя, оценивает количество откладываний дел, восстанавливает последнее корректное значение и сообщает об ошибочном вводе. Архитектура соответствует MVC с активной моделью.

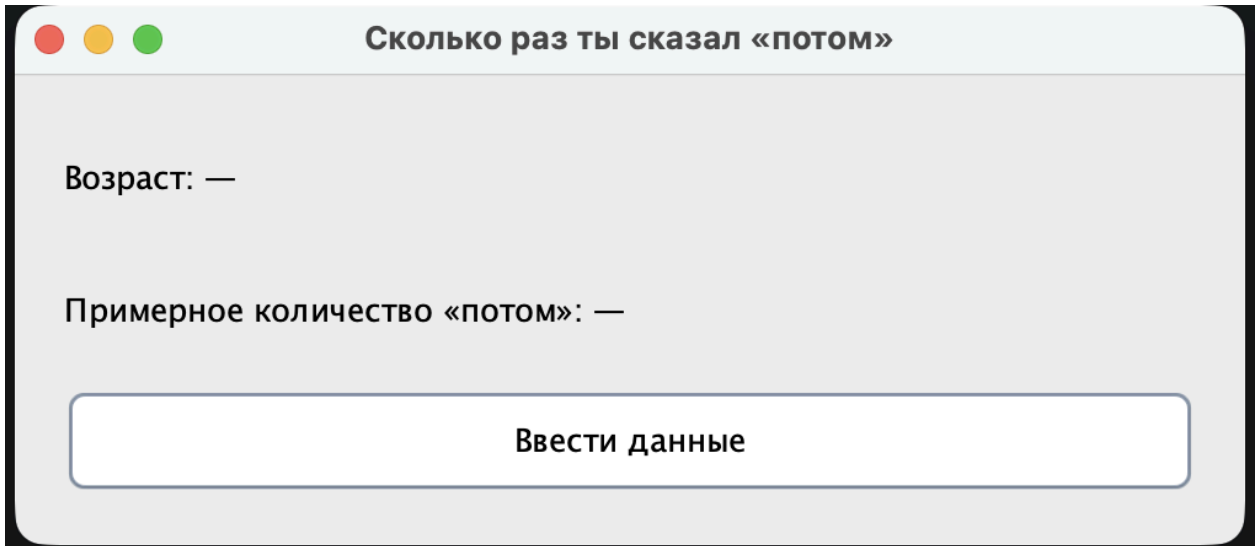


Рисунок 4.1 – Главная форма после запуска

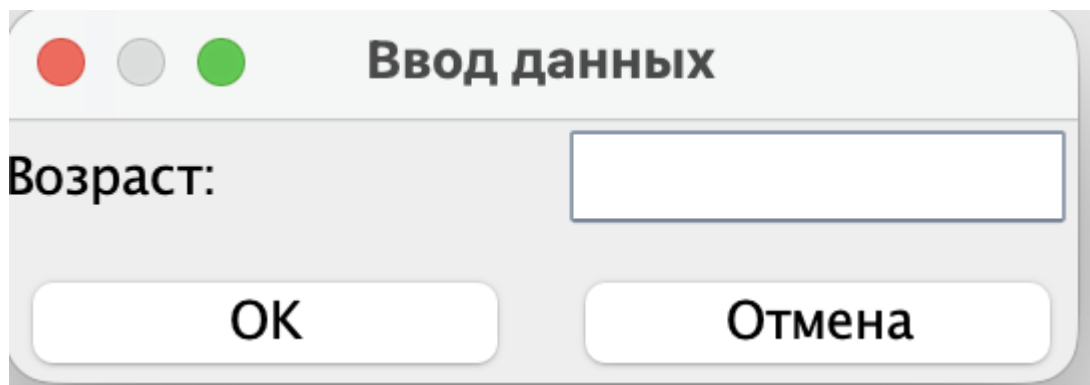


Рисунок 4.2 – Окно ввода данных

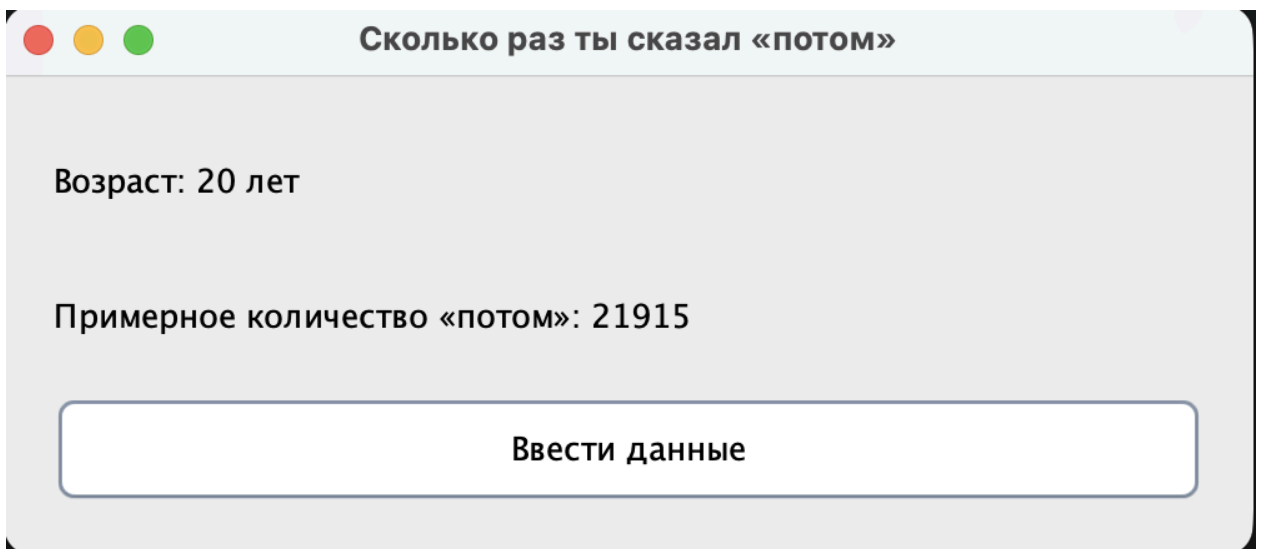


Рисунок 4.3 – Результат вычисления

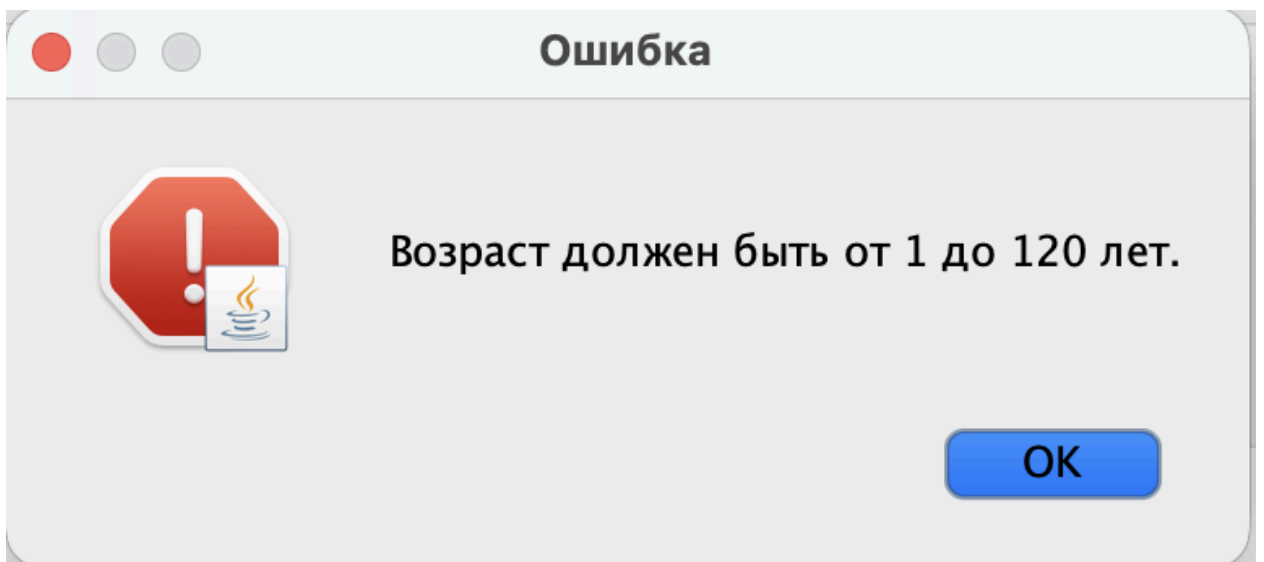


Рисунок 4.4 – Сообщение об ошибке

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, средства навигации и рефакторинга, система сборки и запуска, отладчик, терминал, интеграция с системой контроля версий и механизм расширений.

Вопрос 2. Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение, автоматический импорт, форматирование, быстрые исправления, переход к объявлению, поиск использований, переименование и анализ предупреждений.

Вопрос 3. Каким образом выполняется сборка и запуск проекта?

В IntelliJ IDEA Java-проект можно собрать командой Build Project. Для запуска класса с методом main используется Run. Для отладки применяется Debug, при этом программа может останавливаться в заданных точках останова.

Вопрос 4. Что такое MVC?

MVC – архитектурный паттерн, разделяющий модель данных и предметную логику, представление графического интерфейса и контроллер, обрабатывающий действия пользователя. В данной работе моделью является PostponementModel, представлением – MainView и InputDialog, контроллером – PostponementController.

Вопрос 5. Чем отличается активная модель от пассивной?

Пассивная модель сама не сообщает представлению об изменениях, поэтому контроллер должен отдельно инициировать обновление интерфейса. Активная модель публикует событие при изменении состояния. В данной работе PostponementModel использует PropertyChangeSupport, а MainView получает PropertyChangeEvent и автоматически обновляет элементы интерфейса.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности IntelliJ IDEA при разработке Java-приложения с графическим интерфейсом. Освоены средства редактирования, сборки, запуска и отладки программы, а также рассмотрены возможности подключения средств UML и работы с модельными описаниями.

Была разработана утилита «Сколько раз ты сказал “потом”». Реализован ввод возраста в отдельном окне, проверка корректности данных, сохранение и восстановление последнего корректного значения и вывод результата на главной форме. Приложение построено по паттерну MVC с активной моделью: PostponementModel уведомляет MainView об изменении состояния посредством PropertyChangeSupport.

Поскольку методические указания не задают конкретное значение усреднённой статистики, в работе явно принято учебное допущение о трёх откладываниях в день. Это позволяет получить воспроизводимый расчёт и продемонстрировать работу архитектуры приложения.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода

Main.java

```
import javax.swing.SwingUtilities;
public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            PostponementModel model = new PostponementModel();
            MainView view = new MainView();
            new PostponementController(model, view);
            view.setVisible(true);
        });
    }
}
```

InputDialog.java

```
import javax.swing.*;
import java.awt.*;

public class InputDialog extends JDialog {
    private final JTextField ageField = new JTextField(10);
    private boolean confirmed;

    public InputDialog(Frame owner, int previousAge) {
        super(owner, "Ввод данных", true);
        ageField.setText(previousAge > 0 ? String.valueOf(previousAge) : "");

        JButton okButton = new JButton("OK");
        JButton cancelButton = new JButton("Отмена");

        okButton.addActionListener(e -> { confirmed = true; setVisible(false); });
        cancelButton.addActionListener(e -> { confirmed = false;
        setVisible(false); });

        JPanel form = new JPanel(new GridLayout(2, 2, 8, 8));
        form.add(new JLabel("Возраст:"));
        form.add(ageField);
        form.add(okButton);
        form.add(cancelButton);

        add(form);
        pack();
        setLocationRelativeTo(owner);
    }

    public boolean isConfirmed() { return confirmed; }
    public int getAge() { return Integer.parseInt(ageField.getText().trim()); }
}
```

MainView.java

```
import javax.swing.*;
import java.awt.*;

public class MainView extends JFrame {
    private final JLabel ageLabel = new JLabel("Возраст: —");
    private final JLabel resultLabel = new JLabel("Примерное количество «потом»: —");
    private final JButton inputButton = new JButton("Ввести данные");
```

```

public MainView() {
    setTitle("Сколько раз ты сказал «потом»");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setSize(500, 220);
    setLocationRelativeTo(null);

    JPanel panel = new JPanel(new GridLayout(3, 1, 10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
    panel.add(ageLabel);
    panel.add(resultLabel);
    panel.add(inputButton);
    add(panel);
}

public JButton getInputButton() { return inputButton; }

public void updateResult(int age, long result) {
    ageLabel.setText("Возраст: " + age + " лет");
    resultLabel.setText("Примерное количество «потом»: " + result);
}

public void showError(String message) {
    JOptionPane.showMessageDialog(this, message, "Ошибка",
JOptionPane.ERROR_MESSAGE);
}
}

```

PostponementContriller.java

```

import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;

public class PostponementController implements PropertyChangeListener {
    private final PostponementModel model;
    private final MainView view;

    public PostponementController(PostponementModel model, MainView view) {
        this.model = model;
        this.view = view;
        model.addPropertyChangeListener(this);
        view.getInputButton().addActionListener(e -> openInputDialog());
    }

    private void openInputDialog() {
        InputDialog dialog = new InputDialog(view, model.getAge());
        dialog.setVisible(true);
        if (!dialog.isConfirmed()) return;

        try {
            model.setAge(dialog.getAge());
        } catch (NumberFormatException e) {
            view.showError("Введите корректное целое число.");
        } catch (IllegalArgumentException e) {
            view.showError(e.getMessage());
        }
    }

    @Override
    public void propertyChange(PropertyChangeEvent evt) {
        view.updateResult(model.getAge(), model.getEstimatedPostponements());
    }
}

```

PostponementModel.java

```
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

public class PostponementModel {
    private final PropertyChangeSupport pcs = new PropertyChangeSupport(this);
    private int age;
    private long estimatedPostponements;

    public void setAge(int age) {
        if (age < 1 || age > 120) {
            throw new IllegalArgumentException("Возраст должен быть от 1 до 120 лет.");
        }
        int oldAge = this.age;
        long oldResult = this.estimatedPostponements;
        this.age = age;
        this.estimatedPostponements = Math.round(age * 365.25 * 3);
        pcs.firePropertyChange("age", oldAge, this.age);
        pcs.firePropertyChange("estimatedPostponements", oldResult,
this.estimatedPostponements);
    }

    public int getAge() { return age; }
    public long getEstimatedPostponements() { return estimatedPostponements; }

    public void addPropertyChangeListener(PropertyChangeListener listener) {
        pcs.addPropertyChangeListener(listener);
    }
}
```